Generalizable Lifelong Model Editing via Preference Optimization

Dahyun Jung **Suhyune Son** **Heuiseok Lim***
Department of Computer Science and Engineering, Korea University
{dhaabb55,ssh5131,limhseok}@korea.ac.kr

Abstract

Knowledge editing enables rapid updates of specific factual knowledge in large language models (LLMs) without full retraining. However, more realistic scenarios call for a lifelong framework that handles continual updates rather than one-off modifications. In such settings, existing editing methods often overfit to target prompts, significantly degrading both the generalization of the edited knowledge and the model's general capabilities. To address this issue, we propose GLIME (Generalizable Lifelong Model Editing), which combines knowledge editing with preference optimization over generation behavior. GLIME further incorporates replay-based editing and a gradient constraint to preserve previously edited knowledge. Experimental results show that GLIME significantly improves knowledge generalization in lifelong editing settings while maintaining both editing performance and general capabilities.[1]

1 Introduction

Large language models (LLMs) acquire extensive world knowledge and strong linguistic capabilities through pretraining (Haviv et al., 2023; OpenAI, 2023; Zhao et al., 2025), but their knowledge must be continually updated to reflect a changing world (Cao et al., 2021; Hartvigsen et al., 2023; Shi et al., 2024). As a practical alternative to prohibitively expensive full retraining, knowledge editing has emerged as a promising approach for updating specific facts by directly modifying model parameters (Mitchell et al., 2022b). However, in real-world scenarios, knowledge updates arrive not as isolated events but as an ongoing stream, requiring models to support lifelong knowledge editing under continual update requests (Hartvigsen et al., 2023).

In this lifelong setting, existing knowledge editing methods face two major limitations. First, they often achieve high editing success only on prompts that are lexically similar to the edited input, while generalizing poorly to novel contexts that require the edited knowledge (Meng et al., 2023a; Zhong et al., 2023; Yang et al., 2025). This suggests that models tend to overfit to shallow lexical patterns surrounding specific triggers rather than internalizing the edited knowledge (Li et al., 2024; Ju et al., 2024; Thede et al., 2025). Second, as edits accumulate, parameter updates gradually drift away from the original model distribution, thereby degrading the model's general capabilities (Li and Chu, 2024; Gupta et al., 2024a). Such distributional shifts compound over time, eventually leading to a severe collapse in overall quality.

A primary underlying cause of this issue is that existing studies focus primarily on superficial objectives, such as edit success and locality on unrelated queries (Wang et al., 2024a,c). While these objectives are necessary, they do not ensure the generalization of edited knowledge or the preservation of the model's general capabilities. Consequently, methods that appear effective under restricted evaluation settings suffer severely from collapse due to overfitting as edits accumulate (Yang et al., 2025).

In this paper, we propose **GLIME (Generalizable Lifelong Model Editing)**, a framework designed to address the challenges of lifelong knowledge editing. As shown in Figure 1, GLIME treats continual editing not as a single-objective problem of merely increasing the probability of target tokens, but as a joint optimization problem that both injects new knowledge and preserves the model's generation capabilities. To this end, GLIME integrates preference optimization into the editing process to maintain the model's general capabilities, thereby mitigating overfitting to target texts and facilitating generalized editing. Moreover, GLIME mitigates the catastrophic forgetting (Gupta et al.,

*Corresponding author.
[1]Our code is available at https://github.com/ekgus9/GLIME.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 3899–3920
October 24-29, 2026 ©2026 Association for Computational Linguistics

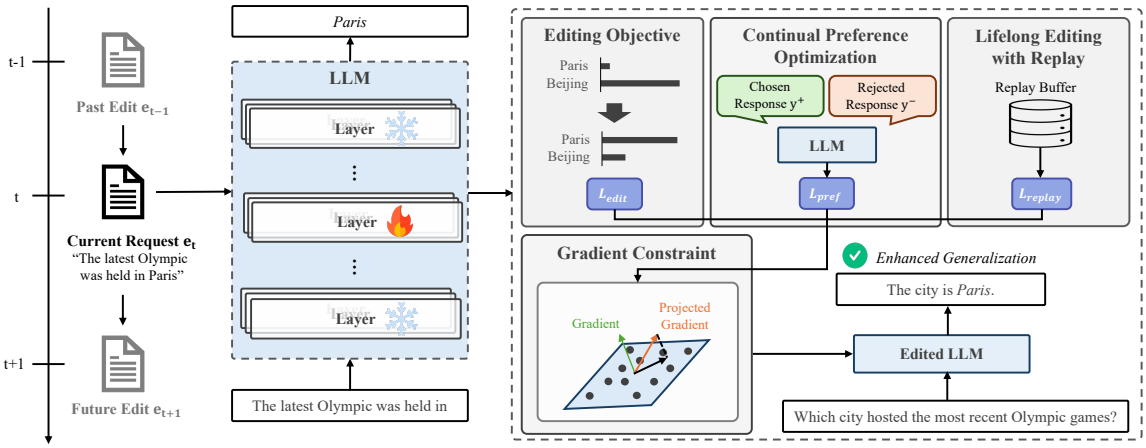


Figure 1: Overall architecture of GLIME. *Left*: The model is updated over a stream of continual knowledge edits using new knowledge injection, preference optimization for behavior-level regularization, and experience replay to prevent catastrophic forgetting. The gradient is further projected orthogonally to previous update directions to mitigate long-term collapse under continual edits. *Bottom right*: The final edited model successfully answers paraphrased queries under autoregressive decoding, indicating improved generalization.

2024b; Luo et al., 2025) of past edits through two strategies. First, replay-based editing helps the model retain previously edited knowledge over time. Second, an orthogonal basis constraint prevents gradient updates for new edits from interfering with the feature subspace associated with past edits, thereby structurally protecting previously edited knowledge from being distorted.

To better assess the generalization of GLIME, we adopt a more realistic evaluation setting. Existing studies predominantly rely on teacher forcing-based evaluations, which tend to overestimate the model's generative capabilities by exposing the ground-truth targets (Yang et al., 2025). We therefore evaluate edited knowledge under autoregressive decoding, which more faithfully reflects its practical use. Experiments show that, compared with existing editing methods, GLIME significantly improves the generalization of edited knowledge, even with a large number of accumulated edits (e.g., 5k), while preserving editing performance and effectively mitigating model degradation. Our main contributions are as follows:

- We formulate lifelong knowledge editing as a problem of jointly integrating new knowledge, generalizing edited knowledge, and preserving the model's general capabilities.
- The proposed GLIME combines target editing with preference optimization to preserve general generation quality and reduce overfitting, while employing replay and gradient-space constraints to alleviate catastrophic forgetting.
- We introduce an autoregressive generation-based evaluation setting and show that GLIME improves the generalization and stability of lifelong editing.

2 Related Work

Model Editing Model editing updates specific knowledge in a pretrained language model by locally modifying its parameters, avoiding full retraining when correcting errors or incorporating new facts (Zhu et al., 2020; Cao et al., 2021). Early approaches often rely on a small number of fine-tuning steps per edit request (Zhu et al., 2020). Later approaches improve efficiency and scalability through hypernetwork-based editors (Cao et al., 2021; Mitchell et al., 2022a), locate-then-edit methods (Meng et al., 2023a,b), and modular editing with adapters or memory modules (Hartvigsen et al., 2023). ROME (Meng et al., 2023a) edits factual associations by identifying a causal multi-layer perceptron (MLP) module and applying a rank-one update to write a new key–value pair. MEMIT (Meng et al., 2023b) extends this idea to multi-layer updates, enabling large-scale simultaneous editing of many facts.

Lifelong Editing In realistic settings, knowledge updates arrive not as isolated events but as an accumulating stream, requiring models to handle continual edit requests under lifelong editing (Hartvigsen et al., 2023). In this setting, continual edits not only cause forgetting of previously edited knowledge, but also gradually push model

parameters away from the pretrained distribution, leading to model degradation in general capabilities (Gupta et al., 2024a). To address this issue, GRACE (Hartvigsen et al., 2023) introduces a discrete key–value adapter cache that preserves the original model weights while sequentially adding new knowledge. WISE (Wang et al., 2024a) proposes a dual-memory architecture with a main memory for pretrained knowledge and a side memory for edited knowledge, together with a router that selects between them. R-ROME (Gupta et al., 2024a) analyzes the instability of sequential ROME updates and introduces a more stable editing formulation to mitigate model collapse.

Continual Learning Lifelong knowledge editing is closely related to the problems of forgetting and interference in continual learning (McCloskey and Cohen, 1989). To mitigate these issues, prior work has developed methods such as weight-importance regularization (Kirkpatrick et al., 2017), gradient projection and constraints (Lopez-Paz and Ranzato, 2022), and experience replay (Rolnick et al., 2019) to preserve performance on past data. More recent work extends these ideas to LLMs. GORP (Wang et al., 2025) projects continual fine-tuning updates into a unified low-rank gradient subspace across full-rank and low-rank parameters. Abbes et al. (2025) show that experience replay and gradient alignment improve stability in continual LLM pretraining.

Existing model editing methods either remain limited to simple knowledge memorization in constrained settings or fail to prevent model degradation under accumulated edits. In this study, we propose GLIME, which mitigates these issues through preference optimization and mitigates catastrophic forgetting by integrating an orthogonal subspace constraint with replay.

3 Preliminaries

We denote a pretrained causal language model with parameters θ as a conditional distribution $p_\theta(\cdot \mid x)$. Given an input prompt x, the model generates a token sequence $y = (y_1, \ldots, y_{|y|})$ autoregressively:

$$p_\theta(y \mid x) = \prod_{i=1}^{|y|} p_\theta(y_i \mid x, y_{<i}). \tag{1}$$

Model editing aims to locally update knowledge in a pretrained model θ by directly modifying its parameters with respect to a target fact. A single edit request e consists of a query prompt x and a target response y^{new} that the edited model is expected to generate. In template-based settings, the query prompt is typically constructed as $x = \pi(s)$, where $\pi(\cdot)$ is a prompt template and s is the subject. The goal of a single edit is to make the edited model assign higher probability to y^{new} than to the original response y for the query prompt x. The model is updated by an editing algorithm $\mathcal{E}$ as follows:

$$\theta' = \mathcal{E}(\theta, e). \tag{2}$$

In realistic settings, knowledge update requests arrive not as isolated events but as a stream over time. We represent this process as a sequence of edit requests $\{e_t\}_{t=1}^{T}$, where each $e_t = (x_t, y_t^{\text{new}})$ specifies a distinct fact to be updated at time step t. Starting from an initial model θ_0, the editing algorithm updates the parameters sequentially:

$$\theta_t = \mathcal{E}(\theta_{t-1}, e_t), \quad t = 1, \ldots, T. \tag{3}$$

In lifelong editing, four criteria are typically considered when designing an editing algorithm (Zhang et al., 2024; Fang et al., 2025). (i) *Reliability*: the edited model correctly generates the target response y_t^{new} for the requested prompt x_t. (ii) *Generalization*: the edited knowledge is not tied only to the surface form of the original prompt, but can also be applied to paraphrases or queries in novel contexts. This is a key criterion for preventing edits from degenerating into local memorization around specific triggers. (iii) *Locality*: the model's original behavior is preserved on queries x unrelated to the edit. This reflects whether the update remains localized around the target fact while maintaining performance on unrelated inputs. (iv) *General Capabilities*: the extent to which the model preserves its overall language quality and capabilities (e.g., language understanding, generation, reasoning, and instruction following) acquired during pretraining as edits accumulate.

4 Methodology

Our proposed GLIME (Generalizable Lifelong Model Editing) views lifelong knowledge editing not simply as a problem of correctly injecting new facts, but as one of mitigating overfitting to target texts, preventing model degradation under accumulated edits, and preserving past edits over time. Ultimately, GLIME aims to improve the generalization of edited knowledge. To this end, GLIME

integrates three components within a unified editing loop: (i) continual preference optimization, (ii) replay-based continual editing, and (iii) a gradient-space constraint.

4.1 Continual Preference Optimization

To maintain stable output quality under continual edits, GLIME incorporates preference optimization into the editing loop as a behavior-level regularizer. Given an edit request $e = (x, y^{\text{new}})$, we first define the standard editing objective as follows:

$$\mathcal{L}_{\text{edit}}(\theta; e) = -\log p_\theta\big(y^{\text{new}} \mid x\big). \tag{4}$$

However, this objective locally increases the probability of the target answer only for the given prompt, gradually pushing the model away from the original language distribution learned during pretraining as edits accumulate. As a result, the model's general generation ability may deteriorate during actual decoding.

GLIME therefore introduces preference optimization as a behavior-level constraint. Specifically, for a context u, we consider a preference pair consisting of a preferred response y^+ and a dispreferred response y^-. Here, u is sampled from a preference dataset used for model alignment and is independent of the edit target prompt x. These preference pairs serve as a regularization signal that encourages the model to preserve its general preference distribution and generation quality throughout the continual editing process. Because u is independent of the edit trigger, it also acts as an anchor that discourages the model from over-adapting to specific edit requests and distorting its output distribution.

More broadly, jointly optimizing these objectives encourages the model to find parameter updates that satisfy both factual editing and general behavioral consistency. By accommodating multiple objectives simultaneously, the model is guided toward shared effective structures rather than solutions that overfit to a single target, thereby improving generalization (Ruder, 2017).

GLIME minimizes the following logistic preference loss (Rafailov et al., 2024; Hong et al., 2024):

$$\mathcal{L}_{\text{pref}}(\theta; u, y^+, y^-) = -\log \sigma(\Delta_\theta(u)), \tag{5}$$

$$\Delta_\theta(u) = \log p_\theta(y^+ \mid u) - \log p_\theta(y^- \mid u), \tag{6}$$

where $\sigma(\cdot)$ denotes the sigmoid function. This objective encourages the model to assign higher preference to y^+ than to y^- under the context u.

4.2 Replay for Lifelong Memory Retention

In the lifelong setting, optimizing only for the most recent edit e_t degrades performance on previous edits $\{e_i\}_{i<t}$. To mitigate this issue, we maintain a replay buffer $\mathcal{B}_t$ that stores a subset of past edit requests and jointly train on them at the current step. Specifically, at time step t, we add an auxiliary editing loss over past requests sampled from $\mathcal{B}_t$:

$$\mathcal{L}_{\text{replay}}(\theta_t) = \mathbb{E}_{e\sim\mathcal{B}_t}\left[\mathcal{L}_{\text{edit}}(\theta_t; e)\right]. \tag{7}$$

The overall objective optimized by GLIME is then given by:

$$\mathcal{L}(\theta_t) = \mathcal{L}_{\text{edit}}(\theta_t; e_t) + \mathcal{L}_{\text{pref}}(\theta_t) + \lambda_{\text{rep}}\mathcal{L}_{\text{replay}}(\theta_t), \tag{8}$$

where λ_{rep} controls the contribution of the replay loss and is tuned to prevent the model from overfitting excessively to the current edit. Following Meng et al. (2023b); Fang et al. (2025), we restrict editing to the MLP layers that have been identified as playing a key role in factual prediction.

4.3 Preventing Parameter Interference via Orthogonal Basis

Unconstrained parameter optimization causes updates for newly injected knowledge to interfere with the weight subspace used by previous edits, leading to catastrophic forgetting of earlier knowledge. To mitigate this issue, inspired by Farajtabar et al. (2019); Saha et al. (2021), we introduce a gradient-space constraint that preserves update directions important for past edits and restricts new updates to the orthogonal complement of that subspace.

Let $W \in \mathbb{R}^{m\times n}$ denote an editable weight matrix, and let $G_t = \nabla_W \mathcal{L}(\theta_t) \in \mathbb{R}^{m\times n}$ be the gradient of the joint objective at step t. To preserve information from previous edits, the model maintains an orthogonal basis matrix $B_{t-1} \in \mathbb{R}^{n\times r}$, satisfying $B_{t-1}^\top B_{t-1} = I$, which captures the right subspace of accumulated updates from earlier steps. The subspace spanned by B_{t-1}, therefore, represents parameter directions that are important for retaining previously edited knowledge.

To minimize interference with past knowledge, GLIME projects the current gradient G_t onto the orthogonal complement of the existing basis:

$$\tilde{G}_t = G_t(I - B_{t-1}B_{t-1}^\top), \tag{9}$$

where $\tilde{G}_t$ is orthogonal to the previous basis and thus satisfies $\tilde{G}_t B_{t-1} = 0$. As a result, the current update suppresses parameter changes along

directions important for past edits, while encouraging new knowledge to be learned in independent directions.

The basis B_t is updated online, and the detailed update procedure is described in Appendix A. The final one-step parameter update in GLIME is:

$$W \leftarrow W - \eta \tilde{G}_t, \tag{10}$$

where η is the learning rate.

5 Experiments

5.1 Evaluation Metrics

We follow the standard evaluation framework of prior work (Wang et al., 2024a,c), which includes **Reliability**, **Generalization**, and **Locality**. However, conventional teacher forcing (TF)-based evaluation overestimates the usability of edited knowledge by exposing the ground-truth token during generation. To address this issue, we additionally adopt an autoregressive decoding (AD)-based protocol (Yang et al., 2025), in which the model generates the answer without access to the target token. This setting provides a more faithful assessment of how deeply the edited knowledge is integrated into the model's generation process. We further evaluate Portability and General Capabilities. More details on the evaluation metrics are provided in Appendix B[2].

Portability Portability measures whether edited knowledge can be effectively applied to reasoning and downstream tasks, rather than being merely memorized as a direct answer. We consider two reasoning-based evaluations. (i) *Multiple-Choice QA (MC)*: We evaluate whether the model can use edited knowledge to answer multiple-choice questions. Following Su et al. (2024), each question targets an edited fact and provides four answer choices: the pre-edit answer, the post-edit answer, an unrelated answer, and an uncertain option. (ii) *Multi-hop Reasoning QA (MR)*: We evaluate whether the model can use edited knowledge in multi-hop QA, following prior work (Zhong et al., 2023, 2025), and report accuracy.

General Capabilities An editing method should preserve the model's pretrained general capabilities after updates. To evaluate this property, we use five benchmarks: Winogrande (Sakaguchi et al., 2019), ARC (Clark et al., 2018), MathQA (Amini et al., 2019), SQuADv2 (Rajpurkar et al., 2018), and IFEval (Zhou et al., 2023). All benchmarks are evaluated using the Language Model Evaluation Harness (Gao et al., 2024)[3].

5.2 Datasets

We evaluate editing accuracy on MQuAKE-Remastered (Zhong et al., 2025) and ZSRE (Levy et al., 2017). MQuAKE-Remastered is a corrected version of MQuAKE (Zhong et al., 2023), a dataset for multi-hop knowledge editing. From this benchmark, we use the CF-3k split. We perform edits using the original cloze prompts and use paraphrased question-style prompts to evaluate Generalization. For Locality, we select questions that do not overlap with those used for editing and construct a final test set of 1,000 samples. ZSRE is a context-free QA dataset designed for zero-shot relation extraction. After deduplicating the test set, we use 743 examples for evaluation. For preference optimization, we use OpenHermesPreferences (Huang et al., 2024). More details on the datasets are provided in Appendix C.

5.3 Implementation Details

We conduct experiments on two instruction-tuned LLMs from different model families: LLaMA-3.1-8B-Instruct (Llama Team, 2024) and Qwen2.5-7B-Instruct (Qwen et al., 2025). The corresponding checkpoints are 'meta-llama/Llama-3.1-8B-Instruct' and 'Qwen/Qwen2.5-7B-Instruct', both publicly available on Hugging Face[4]. Following EasyEdit (Wang et al., 2024b), we edit the MLP layers 4, 5, 6, 7, and 8. All experiments are conducted in a lifelong editing setting, where edits are applied one by one over the full dataset, and evaluation is performed after all edits have been completed. We train for 3 epochs with a learning rate of 5×10^{-5} on a single RTX A6000 GPU. We set the basis size k to 256, use 3 replay samples, and set the loss weight to 0.1. Additional experimental details are provided in Appendix D, and hyperparameter sensitivity analyses are presented in Appendix E.3.

5.4 Baselines

FT-L (Zhu et al., 2020) injects target facts by fine-tuning a limited set of layers for each edit request.

[2]To mitigate the limitation of string-based matching in capturing semantic correctness, we provide an LLM-as-a-Judge in Appendix E.2 and a qualitative analysis in Appendix F.

[3]https://github.com/EleutherAI/lm-evaluation-harness

[4]https://huggingface.co/

	MQuAKE									ZSRE						
	Reliability		Generalization		Locality		Portability			Reliability		Generalization		Locality		
Method	TF	AD	TF	AD	TF	AD	MC	MR	AVG	TF	AD	TF	AD	TF	AD	AVG
LLaMA-3.1-8B-Instruct																
FT-L	0.021	0.035	0.012	0.038	0.004	0.000	0.000	0.003	0.016	0.134	0.106	0.114	0.094	0.023	0.000	0.078
R-ROME	0.026	0.074	0.009	0.087	0.005	0.010	0.000	0.008	0.030	0.033	0.004	0.028	0.087	0.000	0.000	0.025
GRACE	0.310	0.035	0.203	0.046	**0.571**	**0.714**	0.010	0.060	0.270	0.378	0.021	0.314	0.014	**0.388**	**0.203**	0.220
MEMIT	0.044	0.014	0.044	0.014	0.027	0.010	0.000	0.020	0.022	0.000	0.000	0.000	0.000	0.000	0.000	0.000
WISE	0.479	0.193	0.433	0.147	0.328	0.121	0.000	0.028	0.243	0.379	0.073	0.366	0.055	0.322	0.012	0.201
AlphaEdit	0.912	0.604	0.497	0.514	0.324	0.209	0.434	0.079	0.499	0.882	0.538	0.820	0.451	0.384	0.187	0.544
GLIME	**0.937**	**0.830**	**0.806**	**0.800**	0.517	0.379	**0.633**	**0.184**	**0.700**	**0.915**	**0.670**	**0.891**	**0.598**	0.324	0.133	**0.589**
Qwen2.5-7B-Instruct																
FT-L	0.057	0.140	0.057	0.124	0.032	0.079	0.000	0.033	0.070	0.094	0.006	0.079	0.003	0.040	0.003	0.037
R-ROME	0.000	0.001	0.000	0.001	0.000	0.001	0.000	0.006	0.000	0.226	0.054	0.217	0.050	0.055	0.003	0.101
GRACE	0.349	0.050	0.179	0.055	0.373	**0.684**	0.009	0.079	0.243	0.414	0.024	0.322	0.020	0.381	0.100	0.210
MEMIT	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.001	0.000
WISE	0.512	0.274	0.466	0.202	0.260	0.124	0.008	0.063	0.264	0.463	0.231	0.443	0.197	0.239	0.040	0.269
AlphaEdit	**0.986**	0.201	0.369	0.389	0.361	0.513	0.192	0.081	0.430	**0.984**	0.354	0.870	0.277	**0.383**	**0.113**	0.497
GLIME	0.958	**0.772**	**0.716**	**0.812**	**0.443**	0.364	**0.475**	**0.168**	**0.649**	0.948	**0.417**	**0.907**	**0.373**	0.369	0.093	**0.518**

Table 1: Lifelong knowledge editing results. We compare different editing methods on the MQuAKE and ZSRE benchmarks. For each dataset, we report Reliability, Generalization, and Locality. We additionally evaluate both teacher forcing (TF) and autoregressive decoding (AD), which reflects real generation settings. On MQuAKE, we further measure Portability using Multiple-Choice QA (MC) and Multi-hop Reasoning QA (MR), and report the overall average (AVG) across the presented metrics. Best results per model group are in **bold**.

R-ROME (Gupta et al., 2024a) is a model editing method designed to reduce the model collapse observed in ROME (Meng et al., 2023a) and improve stability under continual edits. MEMIT (Meng et al., 2023b) extends ROME to enable large-scale knowledge editing. GRACE (Hartvigsen et al., 2023) trains a single layer, stores the resulting edited parameters in memory, and retrieves them when a relevant query is given. WISE (Wang et al., 2024a) introduces a dual-memory parameterization for lifelong editing, consisting of a main memory that stores pretrained knowledge and a side memory that stores edited knowledge. AlphaEdit (Fang et al., 2025) mitigates the interference caused by the perturbations used in locating-then-edit methods by constraining updates in the null space of existing knowledge. All implementations and baseline settings are based on EasyEdit (Wang et al., 2024b).

5.5 Main Results

Table 1 compares performance on MQuAKE-Remastered (MQuAKE) and ZSRE, reporting both TF and AD evaluation. Overall, GLIME achieves the most consistently strong performance across both backbones and both datasets, with particularly notable gains under AD.

Existing editing methods suffer substantial degradation under lifelong editing. FT-L, R-ROME, and MEMIT show generally low performance or rapid collapse under both TF and AD. WISE and GRACE achieve relatively strong results on some metrics, but fail to maintain a balanced trade-off between edit success and transfer performance, resulting in limited average performance. This suggests that existing methods struggle to maintain long-term stability and generalization under continual editing, as inter-edit interference and distribution drift accumulate over time.

TF evaluation tends to overestimate editing performance compared with AD. Across many baselines, high TF scores do not necessarily translate into strong AD performance. On MQuAKE with LLaMA-3.1-8B-Instruct, AlphaEdit achieves a high Reliability score of 0.912 under TF, but this drops sharply to 0.604 under AD. A similar pattern is observed on Qwen2.5-7B-Instruct, where performance appears overly tied to the edited prompt and degrades substantially under autoregressive generation. This gap highlights the difference between predicting the correct target token under TF and consistently applying the edited knowledge during actual generation. In the lifelong setting, this discrepancy becomes even more pronounced as edits accumulate, underscoring the importance of AD for

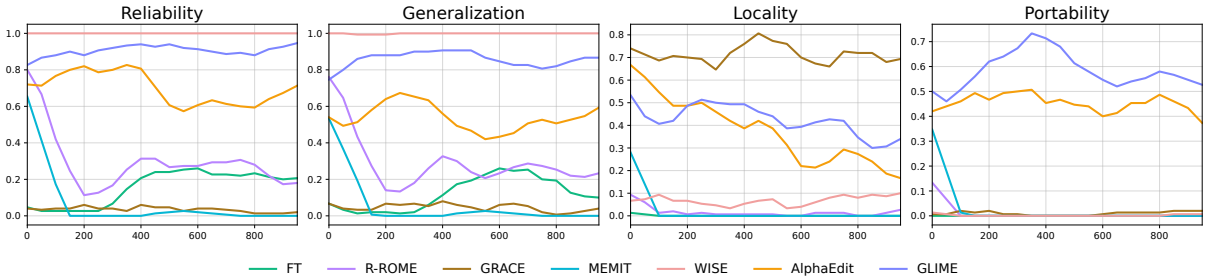


Figure 2: Performance trends under lifelong editing as the number of edits increases. The figure compares performance degradation under accumulated edits and highlights differences in stability across methods.

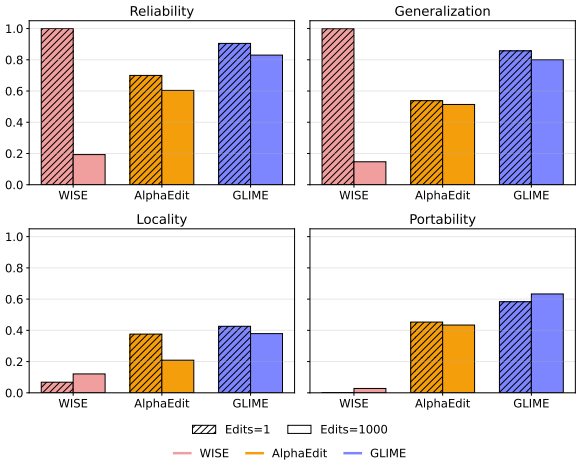


Figure 3: Performance at different evaluation points. We report performance immediately after editing (Edits = 1) and after accumulated edits (Edits = 1000).

evaluating the practical usability of edited knowledge.

GLIME shows the strongest robustness in knowledge generalization and reasoning-based use. GLIME maintains strong Reliability and Generalization on both MQuAKE and ZSRE across both backbones, while also achieving the largest gains on Portability measures that require reasoning. It further preserves Locality without substantial degradation relative to competing methods, alleviating the trade-off between edit success and non-target preservation. These results show that GLIME goes beyond producing the correct answer for the original edit prompt: it enables the model to apply edited knowledge more reliably during generation while mitigating overall performance degradation under accumulated edits.

6 Analysis

Unless otherwise stated, all analysis experiments are conducted on MQuAKE using LLaMA-3.1-8B-Instruct and evaluated under AD. For Portability, we report MC scores.

Stability under Lifelong Editing Figure 2 shows the performance trajectories of different methods as the number of edits increases. Evaluation is performed immediately after each edit to identify the point at which model collapse emerges under accumulated updates. FT-L and GRACE exhibit poor generalization of edited knowledge even at an early stage, suggesting they focus too narrowly on the edit prompt itself. R-ROME and MEMIT show strong edit generalization at first, but undergo sharp model collapse as edits accumulate. WISE produces the edited knowledge accurately immediately after editing, but its low Portability indicates limited ability to use that knowledge in downstream reasoning. AlphaEdit remains relatively strong and robust in edit generalization under accumulated edits, but shows a gradual decline in Locality over time. In contrast, GLIME exhibits smaller performance degradation throughout the entire editing sequence and maintains smoother performance curves, indicating more stable knowledge updating even in long edit streams.

Figure 3 compares performance under different evaluation points. Comparing performance immediately after editing with that after many accumulated edits shows that some methods perform well in the single-edit setting but deteriorate substantially as edits accumulate. WISE achieves very high Reliability and Generalization at Edits = 1, but both metrics drop sharply at Edits = 1000, suggesting that continual updates induce substantial model drift. By contrast, GLIME achieves stronger editing performance than competing methods already at Edits = 1, while also preserving knowledge transfer and model stability under accumulated edits. These results show that GLIME is designed not only for strong one-shot editing performance, but

CPO	Replay	GC	Reliability	Generalization	Locality	$\text{Portability}_{\text{MC}}$	$\text{Portability}_{\text{MR}}$
✓	✓	✓	**0.830**	**0.800**	0.379	**0.633**	**0.184**
	✓	✓	0.826 (-0.004)	0.745 (-0.055)	0.187 (-0.192)	0.014 (-0.619)	0.112 (-0.072)
✓		✓	0.657 (-0.173)	0.633 (-0.167)	0.326 (-0.053)	0.531 (-0.102)	0.096 (-0.088)
✓	✓		0.800 (-0.030)	0.786 (-0.014)	0.385 (+0.006)	0.412 (-0.221)	0.098 (-0.086)
✓			0.597 (-0.233)	0.601 (-0.199)	**0.391** (+0.012)	0.530 (-0.103)	0.168 (-0.016)
	✓		0.624 (-0.206)	0.615 (-0.185)	0.137 (-0.242)	0.063 (-0.570)	0.109 (-0.075)
		✓	0.528 (-0.302)	0.533 (-0.267)	0.259 (-0.120)	0.313 (-0.320)	0.120 (-0.064)
			0.158 (-0.672)	0.120 (-0.680)	0.141 (-0.238)	0.098 (-0.535)	0.087 (-0.097)

Table 2: Ablation results for GLIME across all combinations of its core components. Values in parentheses indicate performance changes relative to the full GLIME. CPO denotes Continual Preference Optimization, Replay denotes the replay loss, and GC denotes the gradient-space constraint.

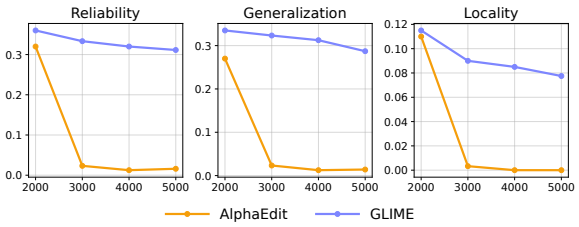


Figure 4: Performance under up to 5,000 edits.

Method	Winogrande	ARC	MathQA	Squadv2	IFEval
Base	0.748	0.802	0.390	0.503	0.532
FT-L	0.248	0.270	0.188	0.002	0.171
R-ROME	0.514	0.234	0.226	0.000	0.229
GRACE	0.748	0.802	0.390	0.503	0.532
MEMIT	0.500	0.244	0.178	0.000	0.184
WISE	0.738	0.778	0.394	0.129	0.209
AlphaEdit	0.716	0.772	0.404	0.654	0.481
GLIME	0.716	0.772	0.412	0.596	0.486

Table 3: General capability results. We evaluate each editing method on representative benchmarks unrelated to the edited knowledge. *Base* denotes the performance of the model before editing, while the other results are measured on the final model after accumulated edits.

Method	FT-L	R-ROME	GRACE	MEMIT	WISE	AlphaEdit	**GLIME**
Seconds	2.574	8.827	5.975	27.826	7.494	16.431	8.936

Table 4: Computational cost comparison. We report the time required for a single edit for each method.

also for robust lifelong editing.

Ablation Results Table 2 summarizes the contribution of each core component of GLIME across all possible combinations. Removing CPO substantially weakens Generalization and Portability. Notably, using Replay and GC without CPO maintains relatively high Reliability and Generalization (0.826 and 0.745, respectively), but results in substantially lower Portability, particularly on MC (0.014). This indicates that these continual learning components mainly contribute to retaining previously edited knowledge, but are insufficient to improve its portability. Removing Replay notably reduces Reliability, indicating weaker retention of previously edited knowledge. Removing GC particularly hurts Portability, suggesting that GC helps prevent interference across edits and preserve the usable integration of edited knowledge. Overall, Replay and GC primarily support the retention of past edits in the lifelong setting, whereas CPO plays a key role in improving the generalization and portability of edited knowledge.

Scaling with Longer Sequences of Edits A lifelong model editing should maintain stable performance as the number of edits increases. To evaluate scalability, we extend the total number of edits to 5,000 and conduct editing on the ZSRE training set. Figure 4 compares GLIME with AlphaEdit, showing that while AlphaEdit suffers a sharp overall performance drop after around 3,000 edits, GLIME remains substantially more stable and scalable throughout the full editing sequence.

Preservation of General Capabilities Table 3 compares performance on downstream benchmarks unrelated to the edited knowledge after editing. FT-L, R-ROME, and MEMIT show substantial degradation on most tasks, suggesting that continual parameter updates accumulate drift from the pretrained distribution and eventually impair the model's overall language and reasoning abilities. GLIME maintains general performance comparable to AlphaEdit and remains stable across diverse tasks, with only small drops relative to the Base model. These results indicate that GLIME is not overly biased toward maximizing edit success alone, but instead effectively mitigates degradation by preserving the model's pretrained general capabilities.

Time and Efficiency Analysis Table 4 reports the time required for a single edit for each method. In lifelong settings, editing must be efficient enough to support rapid updates. GLIME remains practical while achieving competitive efficiency.

7 Conclusion

In this paper, we introduced GLIME, a framework for lifelong knowledge editing in LLMs that addresses superficial overfitting, degradation of general capabilities, and catastrophic forgetting. GLIME combines knowledge injection with preference optimization for preserving generation capabilities, encouraging edited knowledge to generalize beyond specific triggers rather than being shallowly memorized. In addition, it incorporates replay and a gradient-space constraint to mitigate forgetting under accumulated edits. Experiments showed that GLIME degrades more gracefully than strong editing baselines over long edit sequences and enables edited knowledge to transfer more reliably across diverse contexts.

Limitations

GLIME improves generalization and long-term stability under lifelong model editing, but it has several limitations. First, it incurs additional computational and memory overhead by maintaining a replay buffer and a low-rank orthogonal basis of past gradients. Although we validate its effectiveness up to 5,000 edits, real-world deployment may involve much longer edit streams, motivating more efficient replay and basis management. We adopt simple and relatively memory-efficient mechanisms, but exploring more scalable alternatives remains an important direction for future work. Second, its scalability across model sizes and architectures remains unclear. Our experiments are limited to 7B–8B models and specific MLP layers, and future work should examine whether GLIME remains effective in much larger models or architectures such as Mixture-of-Experts (MoE). Finally, our evaluation focuses on mutually unrelated factual knowledge. While MQuAKE and ZSRE allow us to assess explicit factual updates and their transferability, real-world LLM deployment involves more diverse types of knowledge, and the effectiveness of GLIME in such settings remains to be studied.

Acknowledgments

This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(NRF-2021R1A6A1A03045425). This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT) (RS-2024-00398115, Research on the reliability and coherence of outcomes produced by Generative AI). This work was supported by Institute for Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2022-II220369, (Part 4) Development of AI Technology to support Expert Decision-making that can Explain the Reasons/Grounds for Judgment Results based on Expert Knowledge). This work was supported by the Commercialization Promotion Agency for R&D Outcomes(COMPA) grant funded by the Korea government(Ministry of Science and ICT)(2710096072).

References

Istabrak Abbes, Gopeshh Subbaraj, Matthew Riemer, Nizar Islah, Benjamin Therien, Tsuguchika Tabaru, Hiroaki Kingetsu, Sarath Chandar, and Irina Rish. 2025. Revisiting replay and gradient alignment for continual pre-training of large language models. *Preprint*, arXiv:2508.01908.

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. MathQA: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367, Minneapolis, Minnesota. Association for Computational Linguistics.

Nicola De Cao, Wilker Aziz, and Ivan Titov. 2021. Editing factual knowledge in language models. *Preprint*, arXiv:2104.08164.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *Preprint*, arXiv:1803.05457.

Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Shi Jie, Xiang Wang, Xiangnan He, and Tat seng Chua. 2025. Alphaedit: Null-space constrained knowledge editing for language models. *Preprint*, arXiv:2410.02355.

Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. 2019. Orthogonal gradient descent for continual learning. *Preprint*, arXiv:1910.07104.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. The language model evaluation harness.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A survey on llm-as-a-judge. *Preprint*, arXiv:2411.15594.

Akshat Gupta, Sidharth Baskaran, and Gopala Anumanchipalli. 2024a. Rebuilding ROME : Resolving model collapse during sequential model editing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21738–21744, Miami, Florida, USA. Association for Computational Linguistics.

Akshat Gupta, Anurag Rao, and Gopala Anumanchipalli. 2024b. Model editing at scale leads to gradual and catastrophic forgetting. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15202–15232, Bangkok, Thailand. Association for Computational Linguistics.

Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. 2010. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *Preprint*, arXiv:0909.4061.

Tom Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. 2023. Aging with grace: Lifelong model editing with discrete key-value adaptors. *Advances in Neural Information Processing Systems*, 36:47934–47959.

Adi Haviv, Ido Cohen, Jacob Gidron, Roei Schuster, Yoav Goldberg, and Mor Geva. 2023. Understanding transformer memorization recall through idioms. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 248–264, Dubrovnik, Croatia. Association for Computational Linguistics.

Jiwoo Hong, Noah Lee, and James Thorne. 2024. Orpo: Monolithic preference optimization without reference model. *Preprint*, arXiv:2403.07691.

Shengyi Costa Huang, Agustín Piqueres, Kashif Rasul, Philipp Schmid, Daniel Vila, and Lewis Tunstall. 2024. Open hermes preferences. `https://huggingface.co/datasets/argilla/OpenHermesPreferences`.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. Llm-blender: Ensembling large language models with pairwise comparison and generative fusion. In *Proceedings of the 61th Annual Meeting of the Association for Computational Linguistics (ACL 2023)*.

Tianjie Ju, Yijin Chen, Xinwei Yuan, Zhuosheng Zhang, Wei Du, Yubin Zheng, and Gongshen Liu. 2024. Investigating multi-hop factual shortcuts in knowledge editing of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8987–9001.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.

Omer Levy, Minjoon Seo, Eunsol Choi, and Luke Zettlemoyer. 2017. Zero-shot relation extraction via reading comprehension. *arXiv preprint arXiv:1706.04115*.

Qi Li and Xiaowen Chu. 2024. Can we continually edit language models? on the knowledge attenuation in sequential model editing. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5438–5455, Bangkok, Thailand. Association for Computational Linguistics.

Zhoubo Li, Ningyu Zhang, Yunzhi Yao, Mengru Wang, Xi Chen, and Huajun Chen. 2024. Unveiling the pitfalls of knowledge editing for large language models. *Preprint*, arXiv:2310.02129.

Llama Team. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

David Lopez-Paz and Marc'Aurelio Ranzato. 2022. Gradient episodic memory for continual learning. *Preprint*, arXiv:1706.08840.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2025. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *Preprint*, arXiv:2308.08747.

Michael McCloskey and Neal J Cohen. 1989. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pages 109–165. Elsevier.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2023a. Locating and editing factual associations in gpt. *Preprint*, arXiv:2202.05262.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2023b. Mass-editing memory in a transformer. *Preprint*, arXiv:2210.07229.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. 2022a. Fast model editing at scale. *Preprint*, arXiv:2110.11309.

Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D Manning, and Chelsea Finn. 2022b. Memory-based model editing at scale. In *International Conference on Machine Learning*, pages 15817–15831. PMLR.

OpenAI. 2023. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia. Association for Computational Linguistics.

David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. 2019. Experience replay for continual learning. *Advances in neural information processing systems*, 32.

Sebastian Ruder. 2017. An overview of multi-task learning in deep neural networks. *Preprint*, arXiv:1706.05098.

Gobinda Saha, Isha Garg, and Kaushik Roy. 2021. Gradient projection memory for continual learning. *Preprint*, arXiv:2103.09762.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. Winogrande: An adversarial winograd schema challenge at scale. *Preprint*, arXiv:1907.10641.

Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. 2024. Continual learning of large language models: A comprehensive survey. *Preprint*, arXiv:2404.16789.

Zhaochen Su, Jun Zhang, Xiaoye Qu, Tong Zhu, Yanshu Li, Jiashuo Sun, Juntao Li, Min Zhang, and Yu Cheng. 2024. Conflictbank: A benchmark for evaluating the influence of knowledge conflicts in llm. *arXiv preprint arXiv:2408.12076*.

Lukas Thede, Karsten Roth, Matthias Bethge, Zeynep Akata, and Tom Hartvigsen. 2025. Wikibigedit: Understanding the limits of lifelong knowledge editing in llms. *Preprint*, arXiv:2503.05683.

Chenxu Wang, Yilin Lyu, Zicheng Sun, and Liping Jing. 2025. Continual gradient low-rank projection fine-tuning for LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14815–14829, Vienna, Austria. Association for Computational Linguistics.

Peng Wang, Zexi Li, Ningyu Zhang, Ziwen Xu, Yunzhi Yao, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024a. Wise: Rethinking the knowledge memory for lifelong model editing of large language models. *Advances in Neural Information Processing Systems*, 37:53764–53797.

Peng Wang, Ningyu Zhang, Bozhong Tian, Zekun Xi, Yunzhi Yao, Ziwen Xu, Mengru Wang, Shengyu Mao, Xiaohan Wang, Siyuan Cheng, Kangwei Liu, Yuansheng Ni, Guozhou Zheng, and Huajun Chen. 2024b. EasyEdit: An easy-to-use knowledge editing framework for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 82–93, Bangkok, Thailand. Association for Computational Linguistics.

Song Wang, Yaochen Zhu, Haochen Liu, Zaiyi Zheng, Chen Chen, and Jundong Li. 2024c. Knowledge editing for large language models: A survey. *ACM Computing Surveys*, 57(3):1–37.

Wanli Yang, Fei Sun, Jiajun Tan, Xinyu Ma, Qi Cao, Dawei Yin, Huawei Shen, and Xueqi Cheng. 2025. The mirage of model editing: Revisiting evaluation in the wild. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15336–15354, Vienna, Austria. Association for Computational Linguistics.

Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, Siyuan Cheng, Ziwen Xu, Xin Xu, Jia-Chen Gu, Yong Jiang, Pengjun Xie, Fei Huang, Lei Liang, Zhiqiang Zhang, and 3 others. 2024. A comprehensive study of knowledge editing for large language models. *Preprint*, arXiv:2401.01286.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, and 3 others. 2025. A survey of large language models. *Preprint*, arXiv:2303.18223.

Shaochen Zhong, Yifan Lu, Lize Shao, Bhargav Bhushanam, Xiaocong Du, Yixin Wan, Yucheng Shi, Daochen Zha, Yiwei Wang, Ninghao Liu, Kaixiong Zhou, Shuai Xu, Kai-Wei Chang, Louis Feng,

Vipin Chaudhary, and Xia Hu. 2025. MQuAKE-remastered: Multi-hop knowledge editing can only be advanced with reliable evaluations. In *The Thirteenth International Conference on Learning Representations*.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15686–15702, Singapore. Association for Computational Linguistics.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *Preprint*, arXiv:2311.07911.

Chen Zhu, Ankit Singh Rawat, Manzil Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix Yu, and Sanjiv Kumar. 2020. Modifying memories in transformer models. *Preprint*, arXiv:2012.00363.

A Online Update of Basis

This section details how GLIME updates the orthogonal basis memory B_t after each edit step. The goal is to preserve parameter directions that were important for previous edits while allowing the current edit to use only directions in the orthogonal complement.

After the parameter update, we augment the memory with the new right-subspace used by the current edit. A direct SVD of G_t or $\tilde{G}_t$ at every step would be expensive for large layers. Instead, we extract a compact low-rank approximation of the right subspace using a randomized sketch (Halko et al., 2010).

Specifically, let k denote the sketch dimension, and sample a random Gaussian matrix $\Omega_t \in \mathbb{R}^{m\times k}$. We then form the sketch:

$$Y_t = \tilde{G}_t^\top \Omega_t \in \mathbb{R}^{n\times k}. \tag{11}$$

Since Y_t is obtained by multiplying $\tilde{G}_t^\top$ with a random test matrix, the column space of Y_t provides a low-dimensional approximation to the dominant right subspace of the current projected gradient.

Next, we compute a thin-QR factorization:

$$Y_t = Q_t S_t, \tag{12}$$

where $Q_t \in \mathbb{R}^{n\times \hat{r}_t}$ has orthonormal columns, $S_t \in \mathbb{R}^{\hat{r}_t\times k}$, and $\hat{r}_t \le k$ is the numerical rank of the sketch. We use Q_t as the compact basis of the new right-subspace induced by the current edit.

To update the global memory, we concatenate the previous basis and the newly extracted basis:

$$\bar{B}_t = [\, B_{t-1} \;\; Q_t \,]. \tag{13}$$

Although $\tilde{G}_t$ is orthogonal to B_{t-1} in exact arithmetic, a re-orthogonalization step is applied for numerical stability. Concretely, we perform another thin-QR factorization,

$$\bar{B}_t = \hat{Q}_t \hat{S}_t, \tag{14}$$

and set

$$B_t = \hat{Q}_t. \tag{15}$$

By construction, $B_t^\top B_t = I$, and B_t spans both the previously protected right-subspace and the new right directions introduced at step t.

We do not claim that this approach is the only valid choice. We mitigate catastrophic forgetting using simple, relatively memory-efficient replay and gradient-constrained methods. Exploring more sophisticated and efficient approaches would be an important direction for future work.

B Evaluation Metric Details

We adopt four standard knowledge editing metrics, along with a General Capabilities metric, to provide a comprehensive evaluation of lifelong knowledge editing. In particular, to address the limitations of teacher forcing (TF)-based evaluation commonly used in prior work and to better reflect real deployment settings, we treat autoregressive decoding (AD)-based evaluation as a core metric (Yang et al., 2025).

B.1 Difference between TF and AD

Before defining the evaluation metrics, we first clarify the distinction between TF and AD.

Teacher Forcing In TF, when predicting a target sequence token by token, the model predicts the next token $\hat{y}_j$ under the assumption that all previous tokens are given as the ground-truth prefix $y^*_{<j}$. Formally, given a prompt x and a target sequence $y^* = (y^*_1, y^*_2, \ldots, y^*_L)$, the predicted token at each step j is defined as:

$$\hat{y}_j = \arg\max_{v\in\mathcal{V}} p_\theta(v \mid x, y^*_{<j}). \tag{16}$$

Because the input at each step is corrected with the ground-truth token even when the model makes an incorrect prediction at the previous step, distortions in the generation distribution do not accumulate. As a result, TF tends to overestimate the model's actual decoding ability.

Autoregressive Decoding In AD, the model reuses its own previously generated tokens as input at subsequent steps. Given only a prompt x, the generation function $\mathrm{Decode}(\theta, x)$ produces tokens sequentially. Under greedy decoding, the predicted token at step j is defined as:

$$\hat{y}_j = \arg\max_{v\in\mathcal{V}} p_\theta(v \mid x, \hat{y}_{<j}). \tag{17}$$

Unlike TF, AD conditions on the model's own predictions rather than the ground-truth prefix. As a result, a single incorrect token diverts the entire subsequent generation trajectory through $\hat{y}_{<j}$, making AD much more sensitive to exposure bias. This is one reason why existing editing methods often perform well under TF but degrade substantially under AD after aggressive parameter updates. GLIME addresses this issue through a behavior-level constraint based on continual preference optimization.

B.2 Reliability

Reliability measures whether the model can correctly generate the new target response y_t^{new} for a given prompt x_t.

- **TF:** Let L be the length of the target response y_t^{new}. For each target position $j \in \{1, \ldots, L\}$, we condition on the ground-truth prefix $y_{t,<j}^{\text{new}}$ and compare the model's predicted token with the target token $y_{t,j}^{\text{new}}$. Reliability under TF is defined as the average token-level accuracy over the L target positions.

- **AD:** Given only the prompt x_t, we generate an output sequence $\hat{y}_t = \text{Decode}(\theta_t, x_t)$ and evaluate accuracy based on whether the target answer text is exactly contained in the generated output.

B.3 Generalization

Generalization measures whether the edited fact extends beyond surface-level memorization of a specific trigger and can be successfully elicited from a set of K semantically equivalent paraphrased queries, $\mathcal{X}_t^{\text{gen}} = \{x_{t,1}^{\text{gen}}, \ldots, x_{t,K}^{\text{gen}}\}$, under both TF and AD evaluation.

B.4 Locality

Locality measures whether the model preserves its original world knowledge on a set of prompts unrelated to the target fact, $\mathcal{X}_t^{\text{loc}}$, under both TF and AD evaluation.

B.5 Portability

Portability evaluates whether the injected knowledge can be used in downstream tasks or reasoning.

- **Multiple-Choice QA (MC):** We present the model with a four-way multiple-choice question x^{MC} and let it generate the answer directly. We then check whether the generated output, $\text{Decode}(\theta_t, x^{\text{MC}})$, explicitly identifies the target answer option y^{new}.

- **Multi-hop Reasoning QA (MR):** We provide a question x^{MR} in which the edited knowledge must be used as an intermediate step in a reasoning chain, and evaluate accuracy based on whether the final reasoning answer is correctly contained in $\text{Decode}(\theta_t, x^{\text{MR}})$.

B.6 General Capabilities

As knowledge updates accumulate, biased parameter updates may erode the general capabilities that a language model possessed before editing. To assess this effect from multiple perspectives, we evaluate the edited models on five representative NLP benchmarks using the Language Model Evaluation Harness (Gao et al., 2024).

Winogrande (Sakaguchi et al., 2019) Winogrande is a large-scale commonsense reasoning benchmark and an extension of the Winograd Schema Challenge (WSC) for pronoun resolution. We use it to assess whether the model, after knowledge edits, retains the physical and social commonsense needed to correctly infer sentence meaning from context. Accuracy is computed as the fraction of correct options that receive the highest length-normalized log-probability among the candidate choices.

ARC (Clark et al., 2018) ARC consists of grade-school science exam questions, including a challenging subset that cannot be solved by simple pattern matching or surface-level retrieval alone. We use this benchmark to evaluate whether continual forced parameter updates damage the model's underlying scientific reasoning ability and world knowledge.

MathQA (Amini et al., 2019) MathQA is a benchmark for solving complex multi-step math word problems. Although knowledge editing primarily modifies factual associations, severe parameter drift may also harm the structural reasoning pathways required for mathematical problem solving. This benchmark measures the extent to which such degradation affects mathematical reasoning ability.

SQuAD v2 (Rajpurkar et al., 2018) SQuAD v2 evaluates machine reading comprehension on Wikipedia passages. We report the F1 score between the model prediction and the reference answer. Importantly, SQuAD v2 includes unanswerable questions for which no correct answer is present in the provided context. This makes it particularly useful for assessing whether knowledge editing increases hallucination or causes the model to answer too aggressively when it should abstain.

IFEval (Zhou et al., 2023) IFEval measures how strictly a model follows instructions that impose specific formats or constraints. Given a set of

Dataset	Task Category	Input Prompt (x)	Target Output (y)
MQuAKE	**Base Edit** (Rel.)	jazz was created in the country of	Indonesia
	Generalization	Which country was jazz created in?	Indonesia
	Locality	What type of music does Hamid Drake play?	jazz
	Portability (MC)	jazz was created in the country of *Options: (A) Indonesia, (B) uncertain, (C) United States of America, (D) jazz*	(A)
	Portability (MR)	In which country was the music genre played by Hamid Drake created?	Indonesia
ZSRE	**Base Edit** (Rel.)	What programming language was used to write OpenCV?	Java
	Generalization	What is the language of OpenCV?	Java
	Locality	nq question: who was the head of the spanish inquisition	Grand Inquisitor

Table 5: Example knowledge editing instances from the MQuAKE and ZSRE datasets.

	MQuAKE			**ZSRE**		
	Reliability	**Generalization**	**Locality**	**Reliability**	**Generalization**	**Locality**
Target True	0.646	0.418	0.571	0.438	0.428	0.388
Target New	0.200	0.201	0.571	0.331	0.326	0.388

Table 6: Pre-edit performance of LLaMA-3.1-8B-Instruct before editing.

N constraints $\mathcal{C} = \{c_1, \ldots, c_N\}$ specified in the prompt (e.g., starting with a capital letter or avoiding a particular word), we compute the proportion of constraints satisfied by the generated text using strict accuracy. This benchmark verifies whether an instruction-tuned model retains its instruction-following ability after knowledge editing and can continue to respond safely and appropriately to user requests.

C Dataset Details

We evaluate knowledge editing performance using the MQuAKE-Remastered (Zhong et al., 2025) and ZSRE (Levy et al., 2017) datasets, and use OpenHermesPreferences (Huang et al., 2024) as anchor data for continual preference optimization to suppress behavior-level distribution drift. The detailed composition of each dataset and example instances used in the actual benchmark evaluation are described below.

C.1 Knowledge Editing Evaluation Datasets

To compare the editing and retention capabilities of GLIME with those of existing baselines, we adopt MQuAKE-Remastered and ZSRE, two widely used benchmarks for knowledge editing.

MQuAKE-Remastered MQuAKE (Zhong et al., 2023) is originally designed to evaluate whether a single injected fact can transfer successfully through multi-hop reasoning in language models. However, prior work has identified various issues in the original dataset, including logically invalid reasoning chains and incorrect entity mappings. We therefore use MQuAKE-Remastered, a cleaned and improved version of the benchmark. Specifically, we use the CF-3k split, employ cloze prompts for editing and Reliability evaluation, and use paraphrased question-form prompts to evaluate Generalization. Because the dataset does not provide Locality labels, we additionally construct a Locality set of 1,000 samples by selecting questions that do not overlap with those used for editing.

ZSRE ZSRE is a context-free QA dataset originally introduced for zero-shot relation extraction from Wikipedia text, but it has since become a standard benchmark for evaluating whether model editing methods can modify a model's implicit knowledge of specific relational tuples. We preprocess the ZSRE test set by removing duplicate items that may cause editing conflicts, resulting in a final evaluation set of 743 examples.

Table 5 presents example instances from the MQuAKE-Remastered and ZSRE datasets. Table 6

Role	Content
User Query (u)	What's the most reliable medical treatment for premature graying?
Chosen (y^+)	At present, there is no scientifically proven, consistently reliable medical treatment for premature graying, as the causes are often unknown and multifactorial. Graying hair is a natural part of the aging process when the pigment-producing cells in hair follicles slowly reduce their production of melanin, which gives hair its natural color. However, in some cases, premature hair graying can be a result of genetics, stress, certain medical conditions, or other lifestyle factors. If you are concerned about premature graying, it's essential to consult with a healthcare professional, who can help determine the underlying causes and potential treatment options based on your individual case and your specific needs. They may recommend addressing any imbalances in your healthcare plan, improving overall well-being, and monitoring any related medical conditions.
Rejected (y^-)	There is currently no definitive medical treatment to stop or reverse premature graying. The graying of hair is primarily influenced by genetics. However, if there is an underlying medical condition causing premature graying, treating that condition may help slow down the process. It's always best to consult with a healthcare provider or a dermatologist for personalized advice. In some cases, premature graying can be a sign of vitamin B12 deficiency, so addressing this deficiency might help. Certain hair care products and hair dyes can also help to mask the appearance of gray hair. It's important to note that while there are many supplements and home remedies advertised to prevent or reverse graying, there is little scientific evidence to support these claims.

Table 7: Example of preference pairs from the OpenHermesPreferences dataset.

Phase	Template Type	Input Prompt (x)	Target Output (y)
Editing	Context-free cloze	jazz was created in the country of	Indonesia
Evaluation	QA prompt with official chat template	Question: Which country was jazz created in? Answer:	Indonesia

Table 8: Example prompt templates used in the editing and evaluation phases.

reports the TF performance of LLaMA-3.1-8B-Instruct on MQuAKE and ZSRE before any knowledge editing is applied.

- **Target True:** the proportion of cases in which the base model correctly generates the original answer already encoded through pretraining.
- **Target New:** the probability that the model generates the newly injected counterfactual knowledge by chance before editing.

C.2 Preference Dataset

To prevent degradation in generation quality under accumulated edits, GLIME computes a preference optimization loss using edit-independent random contexts u sampled separately from the edit trigger x_t. For this purpose, we use OpenHermesPreferences, a large-scale synthetic preference dataset for alignment. Each instance consists of a user query (u), along with a preferred response (y^+) and a less preferred response (y^-), constructed by ranking generations from multiple LLMs with PairRM (Jiang et al., 2023). Table 7 shows example preference pairs used in training.

D Experimental Setting Details

Additional experimental details and hyperparameter settings not covered in the main text are provided below.

D.1 Algorithms of GLIME

For the pseudocode of GLIME, please refer to Algorithm 1.

D.2 Prompt Templates

We deliberately differentiate the prompt formats used in the editing and evaluation phases. Table 8 shows example prompt templates used in the editing and evaluation phases.

- **Editing Phase:** To directly inject factual knowledge into model parameters, we compute $\mathcal{L}_{\text{edit}}$ using a context-free cloze-style prompt, thereby avoiding unnecessary formatting bias introduced by system prompts or chat templates.

Algorithm 1 GLIME

Require: Initial model θ_0; edit stream $\{e_t = (x_t, y_t^{\text{new}})\}_{t=1}^{T}$; preference dataset $\mathcal{D}_{\text{pref}}$; editable MLP layers $\mathcal{S}$; replay buffer size M; replay weight λ_{rep}; basis size k; epochs E; learning rate η

Initialize replay buffer $\mathcal{B} \leftarrow \emptyset$
for all $\ell \in \mathcal{S}$ **do**
  Initialize orthogonal basis $B^\ell \leftarrow [\,]$
end for
$\theta \leftarrow \theta_0$
for $t = 1$ to T **do**
  for $j = 1$ to E **do**
    Sample a preference pair $(u, y^+, y^-) \sim \mathcal{D}_{\text{pref}}$
    Sample a replay mini-batch $\mathcal{R} \sim \mathcal{B}$
    Compute

$$\mathcal{L} = \mathcal{L}_{\text{edit}}(\theta; e_t) + \mathcal{L}_{\text{pref}}(\theta; u, y^+, y^-) + \lambda_{\text{rep}} \frac{1}{|\mathcal{R}|} \sum_{e \in \mathcal{R}} \mathcal{L}_{\text{edit}}(\theta; e)$$

    for all $\ell \in \mathcal{S}$ **do**
      Compute layer gradient $G^\ell \leftarrow \nabla_{W^\ell} \mathcal{L}$
      if $B^\ell \neq [\,]$ **then**
        Project gradient:

$$\tilde{G}^\ell \leftarrow G^\ell \left(I - B^\ell (B^\ell)^\top \right)$$

      else
        $\tilde{G}^\ell \leftarrow G^\ell$
      end if
      Update parameters:

$$W^\ell \leftarrow W^\ell - \eta \tilde{G}^\ell$$

      Update basis:

$$B^\ell \leftarrow \textsc{UpdateBasis}(B^\ell, \tilde{G}^\ell, k)$$

    end for
  end for
  Update replay buffer:

$$\mathcal{B} \leftarrow \textsc{UpdateBuffer}(\mathcal{B}, e_t, M)$$

end for
return θ

- **Evaluation Phase:** For AD-based evaluation, we generate text using an explicit QA prompt together with the official chat template for each model, so as to better reflect the real usage setting of instruction-tuned language models.

D.3 Decoding Strategy

For AD-based evaluation, all generations are produced using greedy decoding to encourage consistent generation of factual knowledge and control randomness. In addition, given the short-form nature of knowledge editing evaluation, we set max new tokens to 16 to prevent unnecessarily long generations.

D.4 Hyperparameters

We process all knowledge updates sequentially with a batch size of 1. We employ the AdamW optimizer for parameter updates, with the weight decay set to 0. The specific hyperparameter configurations are as follows:

- **Base Hyperparameters:** The learning rate is fixed at 5×10^{-5}, and the number of training epochs per individual edit request is set to 3.
- **Replay Buffer:** For each update, $k = 3$ samples are randomly retrieved from the replay buffer. To prevent overfitting to the current edit, the loss weight for the replay objective, λ_{rep}, is set to 0.1.
- **Gradient Projection:** To operate within practical memory constraints, we maintain a maximum of $r = 4$ basis vectors for the orthogonal gradient projection. When this capacity is exceeded, the basis is updated online using a FIFO (First-In-First-Out) strategy, removing the oldest direction vectors to accommodate new ones.

E More Analysis

E.1 Knowledge Retention

Figure 5 reports per-sample performance on each edit instance e_i, from 1 to 1000 edits, using the final model after all 1,000 edits have been completed. On Reliability and Generalization, AlphaEdit achieves relatively strong performance on recently injected knowledge (edits 800–1000), but exhibits a typical forgetting pattern in which performance drops sharply on older edits (edits 0–200). GLIME shows a similar trend, but maintains consistently higher edit success across the entire sequence. The gap between the two methods becomes even more pronounced on Portability. AlphaEdit remains around 0.4 across nearly all edit instances, suggesting limited ability to transfer edited facts to new contexts. In contrast, GLIME achieves high Portability throughout the full sequence, demonstrating that even older edited facts remain effectively usable in downstream reasoning.

E.2 Semantic Evaluation via LLM-as-a-Judge

AD-based evaluation is straightforward to implement and easy to automate, as it is based on whether

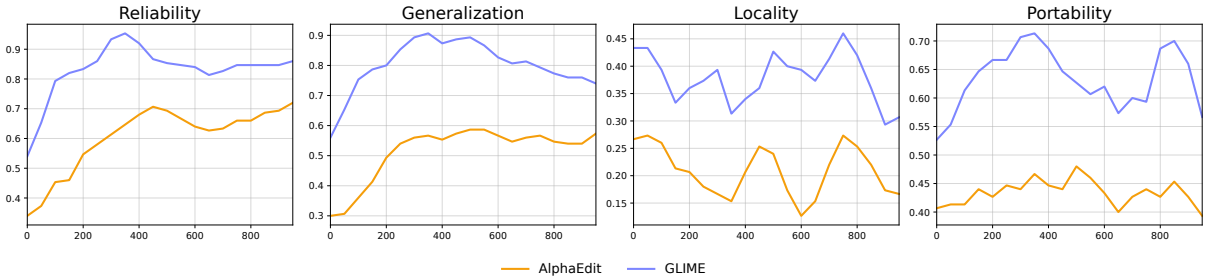


Figure 5: Comparison of model outputs on each sample after 1,000 edits.

You are a strict grader. Given: - Question - Gold target - Predicted answer Return: A if the predicted answer semantically matches the gold target. B otherwise. Only output a single letter: "A" or "B". Question: {question} Gold target: {target} Predicted answer: {predicted_answer}

Table 9: LLM-as-Judge prompt template.

the correct answer is included in the response. However, this approach may not fully capture semantic correctness, since the incidental inclusion of the answer string can still be counted as correct. To address this limitation, we additionally employ an LLM-as-a-Judge (Gu et al., 2025) evaluation for AD responses. We use the instruction prompt shown in Table 9 and employ GPT-4o (OpenAI, 2023) as the judge model. Table 10 reveals a clear difference in performance retention between the two methods across evaluation protocols. Under LLM-as-a-Judge evaluation, which assesses strict semantic correctness, AlphaEdit continues to struggle, whereas GLIME preserves relatively strong performance. These results indicate that the high AD scores of GLIME do not merely arise from malformed outputs that happen to contain the correct answer token, but instead reflect the generation of contextually appropriate and semantically correct responses.

E.3 Sensitivity Analysis of Hyperparameters

To identify an effective configuration for GLIME, we conduct a sensitivity analysis over its key hyperparameters. Table 11 shows how performance varies with changes in the number of training epochs, replay batch size, replay loss weight (λ_{rep}), orthogonal basis size (k), and random seed. The default setting is training epoch $= 3$, batch size $= 3$, $\lambda_{rep} = 0.1$, $k = 256$, and seed $= 7$.

- **Training Epochs:** As the number of training epochs increases from 1 to 5, accumulated parameter updates gradually improve target knowledge injection and reasoning-based transfer. However, this improvement comes with a clear trade-off: Locality, which reflects preservation of existing knowledge, declines from 0.436 to 0.288.

- **Replay Batch Size & Loss Weight (λ_{rep}):** These results reflect the role of experience replay in preserving previously edited knowledge. As the replay batch size increases, edit success improves slightly, but at the cost of higher computational overhead. In particular, as the replay loss weight increases, edit success improves, but Locality declines, suggesting a trade-off caused by overly strong retention of edited knowledge.

- **Basis Size (k):** The basis size determines how sufficiently important update directions from past edits can be captured and preserved. When the basis size is excessively small, such as 32, it fails to adequately represent the principal subspace of previous edits, weakening the interference suppression effect and causing a substantial overall performance drop. In contrast, when the basis size reaches around 256, the evaluation metrics begin to converge, indicating the most stable generalization performance.

- **Gradient Rank (r):** The gradient rank controls the number of directions retained for each gradient subspace. Reducing r from 4

Method	Reliability			Generalization			Locality		
	TF	AD	LLM	TF	AD	LLM	TF	AD	LLM
AlphaEdit	0.912	0.604	0.594	0.497	0.514	0.500	0.324	0.209	0.173
GLIME	**0.937**	**0.830**	**0.775**	**0.806**	**0.800**	**0.746**	**0.517**	**0.379**	**0.328**

Table 10: Comparative analysis across evaluation metrics, including LLM-as-a-Judge.

	Reliability	Generalization	Locality	Portability
Default	0.830	0.800	0.379	0.633
Training Epoch				
1	0.653	0.665	0.436	0.507
2	0.770	0.744	0.350	0.555
4	0.796	0.798	0.308	0.608
5	0.829	0.811	0.288	0.631
Replay Batch Size				
1	0.767	0.751	0.341	0.568
5	0.830	0.817	0.367	0.518
7	0.851	0.843	0.351	0.570
Replay Loss Weight (λ_{rep})				
0.05	0.743	0.681	0.096	0.182
0.20	0.824	0.814	0.298	0.575
Basis Size (k)				
32	0.614	0.530	0.077	0.216
64	0.782	0.770	0.350	0.629
128	0.826	0.809	0.332	0.619
512	0.819	0.822	0.345	0.598
Gradient Rank (r)				
2	0.832	0.818	0.425	0.534
Seed				
42	0.834	0.795	0.400	0.579

Table 11: Sensitivity analysis results of GLIME under different hyperparameter settings.

to 2 slightly improves Reliability, Generalization, and Locality, but decreases Portability from 0.633 to 0.534. This suggests that a smaller rank may insufficiently preserve important update directions from earlier edits, thereby weakening the protection of transferable edited knowledge. We therefore use $r = 4$ as the default setting to maintain a better balance across editing metrics.

- **Seed:** To verify robustness to random initialization, we compare performance by changing the default seed from 7 to 42. The results show that Reliability and Generalization remain at similar levels with little difference, while Locality slightly improves and Portability decreases marginally. Although some variation is observed in individual metrics, the overall performance trend remains unchanged, indicating that GLIME is relatively stable with respect to changes in the random seed.

Method	Reliability	Generalization	Locality	$\text{Portability}_{\text{MC}}$	$\text{Portability}_{\text{MR}}$
Sketch	0.830	0.800	0.379	0.633	**0.184**
SVD	**0.871**	**0.838**	**0.394**	**0.682**	0.181

Table 12: Comparison between the randomized sketch and exact SVD for the gradient-space constraint (GC).

Method	Reliability	Generalization	Locality	$\text{Portability}_{\text{MC}}$	$\text{Portability}_{\text{MR}}$
AlphaEdit	0.604	0.514	0.209	0.434	0.079
+ Replay	0.530	0.545	0.166	0.320	0.085

Table 13: Effect of applying memory replay to AlphaEdit.

E.4 Comparison with Exact SVD

To examine the trade-off introduced by the sketch approximation in GC, we replace the randomized sketch with an exact SVD basis while keeping all other settings unchanged. As shown in Table 12, using exact SVD improves most metrics, increasing the average score from 0.565 to 0.593. However, constructing the exact SVD basis requires approximately $6\times$ more computation time than our sketch-based approach. Since lifelong model editing requires repeated and efficient updates as new edits arrive, this additional computational cost becomes substantial over long edit sequences. We therefore adopt the randomized sketch in GLIME as a practical trade-off, retaining competitive editing performance while substantially reducing the computational overhead of GC.

E.5 Effect of Memory Replay

To examine whether GLIME's gains primarily stem from memory replay, we also apply the same replay strategy to AlphaEdit. As shown in Table 13, Replay slightly improves Generalization from 0.514 to 0.545, but degrades Reliability, Locality, and $\text{Portability}_{\text{MC}}$, resulting in a lower overall average score. These results indicate that replay alone is insufficient for effective lifelong editing. Rather, the improvements of GLIME arise from the complementary interaction between replay-based knowl-

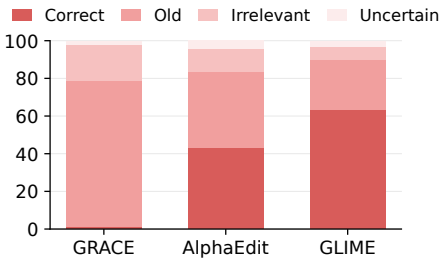


Figure 6: Comparison of answers on the multiple-choice task after editing. *Correct* denotes the proportion of selections corresponding to the edited knowledge, *Old* denotes selections corresponding to the pre-edit knowledge, *Irrelevant* denotes selections of unrelated answers, and *Uncertain* denotes selections of the uncertainty option.

edge retention and the other components of our framework.

E.6 Multiple-Choice Answer Distribution

Figure 6 compares the distribution of answers selected by each model on the multiple-choice task after editing. GRACE shows a strong tendency to rely on pre-edit knowledge, with most responses remaining in the *Old* category, indicating limited transfer of edited facts to downstream decision-making tasks. AlphaEdit partially improves the *Correct* ratio, but still retains a substantial proportion of *Old* responses, suggesting that the use of edited knowledge is not yet stably established. In contrast, GLIME substantially reduces the proportion of *Old* responses, showing that the model goes beyond producing the correct answer only for the edit prompt and instead applies the updated knowledge more consistently in decision-oriented queries. Moreover, the proportions of *Irrelevant* and *Uncertain* responses do not increase excessively, suggesting that GLIME also suppresses undesirable side effects such as random guessing and response uncertainty during editing.

E.7 Loss Scale Analysis

Since GLIME jointly optimizes multiple objectives, we examine whether differences in the magnitudes of their losses cause an imbalance during optimization. Table 14 reports the average magnitude of each objective across training epochs. Although $\mathcal{L}_{edit}$ is substantially larger than $\mathcal{L}_{pref}$ and $\mathcal{L}_{replay}$ in the first epoch, it rapidly decreases as training proceeds and becomes comparable to the other objectives by the third epoch. This indicates that the initial scale discrepancy does not persist throughout optimization.

Epoch	$\mathcal{L}_{edit}$	$\mathcal{L}_{pref}$	$\mathcal{L}_{replay}$
1	2.8943	0.6979	0.5638
2	1.1538	0.5074	0.5545
3	0.5844	0.4319	0.5439

Table 14: Average magnitude of each training objective across epochs.

To further examine whether explicitly compensating for the larger initial magnitude of $\mathcal{L}_{edit}$ is beneficial, we reduce its weight from the default value of 1.0 to 0.5. This results in lower Reliability (0.830 to 0.813), Generalization (0.800 to 0.777), and Portability (0.633 to 0.570), while Locality increases from 0.379 to 0.535. The overall average score also decreases from 0.565 to 0.558. These results suggest that explicitly down-weighting $\mathcal{L}_{edit}$ does not improve the overall balance among the objectives, supporting our default weighting scheme.

F Qualitative Analysis

To provide an intuitive illustration of how GLIME preserves generation quality under lifelong knowledge editing, we present decoding outputs of the language model in Table 15. The example is taken after 1,000 accumulated edits, including an edit that changes the nationality of the target person (Frank R. Strayer) from *American* to *Canadian*. The table shows the model's autonomous generations for the prompts corresponding to each evaluation metric.

- **Model Collapse (FT-L, R-ROME, MEMIT):** FT-L and R-ROME exhibit complete collapse of decoding ability, generating meaningless special symbols (e.g., "://") or corrupted text (e.g., "scalablytyped"). MEMIT also suffers severe collapse under exposure bias, repeatedly generating the same word ("United") without termination.

- **Target Bleeding (WISE):** Although WISE avoids complete collapse through its dual-memory architecture, it excessively amplifies the probability of the target token ("Canada"), causing knowledge bleeding in which the model outputs "Canada" even for prompts about unrelated historical facts.

- **Refusal & Hallucination (GRACE, AlphaEdit):** GRACE and AlphaEdit preserve fluency relatively well, but fail to integrate

the edited knowledge into the reasoning process and instead refuse to answer. In particular, AlphaEdit also exhibits hallucination on the Locality prompt by generating an irrelevant historical fact.

- **GLIME:** GLIME preserves language quality through behavior-level preference optimization. Beyond merely recalling the target answer, it integrates "Canada" into a complete and natural sentence, while also demonstrating robust performance on both multi-hop reasoning and unrelated factual knowledge.

Method	Predicted Answers
[Reliability]	
Prompt: "Frank R. Strayer is a citizen of"	**Target:** Canada
FT-L	://:// of://://:// of:// of ✗ *(Model Collapse)*
R-ROME	INTERRUPTION INTERRUPTION Ras INTERRUPTIONdig... ✗ *(Model Collapse)*
GRACE	I do not have information on a person named Frank R. Strayer. ✗ *(Refusal)*
MEMIT	United United United United United United... ✗ *(Repetition)*
WISE	Canada Canada Canada Canada Canada Canada... ✗ *(Repetition)*
AlphaEdit	I don't have information on Frank R. Strayer. ✗ *(Refusal)*
GLIME	**Canada.** ✓
[Generalization]	
Prompt: "What is the country of citizenship of Frank R. Strayer?"	**Target:** Canada
FT-L	:// of:// of://:// of:// of:// ✗
R-ROME	dyst pers**reeze** INTERRUPTION INTERRUPTION.scalablytyped... ✗
GRACE	I do not have information on the country of citizenship of Frank R. Strayer ✗
MEMIT	United United United United United United... ✗
WISE	Canada Canada Canada Canada Canada Canada... ✗
AlphaEdit	I don't have information on Frank R. Strayer. ✗
GLIME	**Frank R. Strayer is a citizen of Canada.** ✓
[Locality]	
Prompt: "Who founded Tudor dynasty?"	**Target:** Henry VII *(Pre-trained knowledge)*
FT-L	://://://:// of:// of:// of ✗
R-ROME	.scalablytyped.scalablytyped.scalablytyped.scalablytyped... ✗
GRACE	The Tudor dynasty was founded by Henry Tudor, also known as Henry VII ✓
MEMIT	United United United United United United... ✗
WISE	Canada Canada Canada Canada Canada Canada... ✗ *(Knowledge Bleeding)*
AlphaEdit	The founder of the Tudor dynasty was John of Ga. ✗ *(Hallucination)*
GLIME	**The Tudor dynasty was founded by Henry Tudor, who was born in the...** ✓
[Multi-hop Reasoning / Portability]	
Prompt: "Who is the current head of the government in the country of citizenship of Frank R. Strayer?"	
Target: Justin Trudeau *(Canada's head of government)*	
FT-L	of:// of://://:// of://:// of ✗
R-ROME	grisione.scalablytyped.scalablytyped.scalablytyped... ✗
GRACE	I am unable to verify who the current head of government is for Frank R. ✗
MEMIT	United United United United United United... ✗
WISE	Canada Canada Canada Canada Canada Canada... ✗
AlphaEdit	I don't have information on the current head of the government in the country of... ✗
GLIME	**The current head of government in the country of Canada is Justin Trudeau, who is...** ✓

Table 15: Qualitative evaluation of language model generations after accumulated edits.